The second invocation – `getCallback()()` – might look a little funky. If this chapter has made sense so far (I really hope it has!), then it should be obvious that the expression `getCallback()` evaluates to an anonymous function, we can either invoke it immediately or store a reference to it in a variable for future use.

Anonymous Functions and Scope

Earlier we looked at the difference between local and global scope in PHP. If you're coming to PHP from a language that has different scoping rules (such as automatic inheritance of parent scope in JavaScript), then you might find yourself wondering why the following code isn't working:

```
function getCallbackWithArg(string $txt): callable
{
    return function (): void {
        print "You gave me the string '{$txt}'" . PHP_EOL;
    };
}

getCallbackWithArg('I hope this works')(); // spoilers, it doesn't
Output:
Warning: Undefined variable $txt in /usr/src/myapp/anon_func5.php on line 6
You gave me the string ''
```

Yep, definitely broken. Here's an equivalent in JavaScript:

```
function getCallbackWithArg(txt) {
    return function () {
        console.log("You gave me the string '" + txt + "'\n");
    };
}

getCallbackWithArg('I hope this works')();

Output:
You gave me the string 'I hope this works'
```